13601-02, 13601-03: 알고리즘

Spring 2026, School of Computer Science and Engineering

Practice 2 – 탐욕적 알고리즘

Description: 벨만-포드 알고리즘과 플루이드-워셜 알고리즘을 이용해 최단 거리와 경로, 알고리즘 수행시간을 출력합니다. 문제 풀이를 위해 500개의 정점을 가진 희소 그래프가 파일로 주어집니다. 희소 그래프에서 0번 정점과 499번 정점 정점을 출발지와 목적지로 하여 풀이합니다. 경로 출력 시 강의 중 언급한 역추적(Traceback) 방법을 사용합니다.

총 2가지의 결과에 대해 최단 거리와 경로 일치 여부, 알고리즘 수행 시간을 비교 분석하여 제출합니다.

다음은 데이터 파일의 구조입니다. 데이터 파일은 sparse_graph_500.txt 한 가지로 제공됩니다.

〈데이터 파일 구조〉	〈파일 예시〉
	5 6
〈정점수〉 〈간선수〉	0 1 45
〈정점번호〉 〈정점번호〉 〈가중치〉	0 2 12
…	1 3 88
〈정점번호〉 〈정점번호〉 〈가중치〉	2 3 5
	2 4 100
	3 4 50

분량 제한은 없으며 자유 형식으로 제출하시기 바랍니다.

CAUTION: 규격에 맞지 않는 제출물 제출 및 기간 외 제출은 미제출로 간주합니다.

코드 – 프림(배열/우선순위 큐) & 크루스칼 알고리즘

```
import time

INF = float('inf')


# 데이터 파싱 함수
def read_graph_data(filename):
    with open(filename, 'r') as f:
        tokens = list(map(int, f.read().split()))

    V, E = tokens[0], tokens[1]

    edges = []
    idx = 2
    for _ in range(E):
        u = tokens[idx]
        v = tokens[idx + 1]
        w = tokens[idx + 2]
        edges.append((u, v, w))
        idx += 3

    return V, E, edges


# ==========================================
# 1. 벨만-포드 알고리즘: 단일 출발점 최단 경로
# ==========================================
def bellman_ford(V, edges, start, end):
```

```
    dist = [INF] * V
    pred = [-1] * V

    dist[start] = 0

    start_time = time.perf_counter()

    for _ in range(V - 1):
        updated = False

        for u, v, w in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                pred[v] = u
                updated = True

        if not updated:
            break

    end_time = time.perf_counter()

    # 역추적
    path = []
    if dist[end] != INF:
        curr = end

        while curr != -1:
            path.append(curr)
            curr = pred[curr]

        path.reverse()

    return dist[end], path, end_time - start_time


# ==========================================
# 2. 플로이드-워셜 알고리즘: 모든 정점 쌍 최단 경로
# ==========================================
def floyd_warshall(V, edges, start, end):
    dist = [[INF] * V for _ in range(V)]
    nxt = [[-1] * V for _ in range(V)]

    for i in range(V):
        dist[i][i] = 0
        nxt[i][i] = i

    for u, v, w in edges:
        if w < dist[u][v]:
            dist[u][v] = w
            nxt[u][v] = v

    start_time = time.perf_counter()

    for k in range(V):
        for i in range(V):
            if dist[i][k] == INF:
                continue

            for j in range(V):
                if dist[k][j] == INF:
                    continue

                if dist[i][k] + dist[k][j] < dist[i][j]:
```

```
                    dist[i][j] = dist[i][k] + dist[k][j]
                    nxt[i][j] = nxt[i][k]

    end_time = time.perf_counter()

    # 경로 복원
    path = []
    if dist[start][end] != INF:
        curr = start
        path.append(curr)

        while curr != end:
            curr = nxt[curr][end]

            if curr == -1:
                path = []
                break

            path.append(curr)

    return dist[start][end], path, end_time - start_time


# ==========================================
# 메인 실행부
# ==========================================
if __name__ == "__main__":
    filename = "sparse_graph_500-1.txt"
    start_vertex = 0
    end_vertex = 499

    try:
        V, E, edges = read_graph_data(filename)
    except FileNotFoundError:
        print(f"오류: '{filename}' 파일을 찾을 수 없습니다.")
        exit()

    print(f"\n[{filename} 분석 결과]")
    print(f"정점 수: {V}, 간선 수: {E}")
    print(f"출발 정점: {start_vertex}, 도착 정점: {end_vertex}")
    print()

    # 1. 벨만-포드
    bf_dist, bf_path, bf_time = bellman_ford(V, edges, start_vertex, end_vertex)

    print("1. 벨만-포드 알고리즘")
    print(f"   최단 거리: {bf_dist}")
    print(f"   경로: {' -> '.join(map(str, bf_path))}")
    print(f"   수행 시간: {bf_time:.6f} 초")
    print()

    # 2. 플로이드-워셜
    fw_dist, fw_path, fw_time = floyd_warshall(V, edges, start_vertex, end_vertex)

    print("2. 플로이드-워셜 알고리즘")
    print(f"   최단 거리: {fw_dist}")
    print(f"   경로: {' -> '.join(map(str, fw_path))}")
    print(f"   수행 시간: {fw_time:.6f} 초")
    print()
```

```python
# 결과 비교
print("3. 결과 비교")

if bf_dist == fw_dist:
    print("   최단 거리 일치 여부: 일치")
else:
    print("   최단 거리 일치 여부: 불일치")

if bf_path == fw_path:
    print("   경로 일치 여부: 일치")
else:
    print("   경로 일치 여부: 불일치")

if bf_dist == fw_dist and bf_path == fw_path:
    print("\n=> 검증 완료: 두 알고리즘의 최단 거리와 경로가 동일합니다.\n")
else:
    print("\n=> 주의: 최단 거리는 같아도 경로는 다를 수 있습니다.\n")
    print("   최단 경로가 여러 개 존재하면 두 알고리즘이 서로 다른 최단 경로를 출력할 수 있습니다.\n")
```

1번 코드 실행 결과

```
sehn00@sehn-pc:~/algorithm_4_1/hw3
$ python3 bellman_floyd_compare.py

[sparse_graph_500-1.txt 분석 결과]
정점 수: 500, 간선 수: 998
출발 정점: 0, 도착 정점: 499

1. 벨만-포드 알고리즘
   최단 거리: 405
   경로: 0 -> 1 -> 141 -> 97 -> 471 -> 472 -> 473 -> 474 -> 475 -> 336 -> 205 -> 268 -> 499
   수행 시간: 0.000494 초

2. 플로이드-워셜 알고리즘
   최단 거리: 405
   경로: 0 -> 1 -> 141 -> 97 -> 471 -> 472 -> 473 -> 474 -> 475 -> 336 -> 205 -> 268 -> 499
   수행 시간: 3.344579 초

3. 결과 비교
   최단 거리 일치 여부: 일치
   경로 일치 여부: 일치

=> 검증 완료: 두 알고리즘의 최단 거리와 경로가 동일합니다.
```

결과 분석 및 고찰

(1) 벨만-포드 알고리즘의 수행 결과

벨만-포드 알고리즘을 이용하여 희소 그래프에서 0번 정점을 출발지, 499번 정점을 목적지로 설정하고 최단 거리와 경로를 계산하였다. 실행 결과 최단 거리는 405로 나타났으며, 역추적을 통해 구한 최단 경로는 0 -> 1 -> 141 -> 97 -> 471 -> 472 -> 473 -> 474 -> 475 -> 336 -> 205 -> 268 -> 499로 출력되었다. 수행 시간은 약 0.000494초로 측정되었으며, 매우 짧은 시간 안에 결과를 도출하였다.

이는 벨만-포드 알고리즘이 단일 출발점 최단 경로 문제에 특화되어 있기 때문이다. 본 과제에서는 모든 정점 쌍의 최단

거리가 아니라, 0번 정점에서 499번 정점까지의 최단 거리와 경로만 필요하다. 따라서 벨만-포드는 주어진 간선들을 반복적으로 완화하면서 필요한 최단 거리 정보를 계산할 수 있었고, 그래프가 정점 500개, 간선 998개의 희소 그래프였기 때문에 실제로 확인해야 할 간선 수가 많지 않아 매우 빠르게 동작하였다.

(2) 플로이드-워셜 알고리즘의 수행 결과

플로이드-워셜 알고리즘을 이용하여 동일한 그래프에서 0번 정점부터 499번 정점까지의 최단 거리와 경로를 계산하였다. 실행 결과 최단 거리는 벨만-포드 알고리즘과 동일하게 405로 나타났으며, 경로 역시 0 -> 1 -> 141 -> 97 -> 471 -> 472 -> 473 -> 474 -> 475 -> 336 -> 205 -> 268 -> 499로 동일하게 출력되었다. 이를 통해 두 알고리즘이 동일한 최단 거리와 최단 경로를 계산했음을 확인할 수 있었다.

그러나 수행 시간은 약 3.344579초로 측정되어 벨만-포드 알고리즘보다 훨씬 오래 걸렸다. 이는 플로이드-워셜 알고리즘이 특정 출발점과 도착점만을 대상으로 하는 알고리즘이 아니라, 모든 정점 쌍 사이의 최단 거리를 계산하는 알고리즘이기 때문이다. 따라서 실제로 필요한 결과가 0번 정점에서 499번 정점까지의 경로 하나뿐이더라도, 알고리즘 수행 과정에서는 500개의 모든 정점 조합에 대해 중간 정점을 거쳐 가는 경우를 확인해야 한다. 이로 인해 O(V^3)의 시간 복잡도를 가지며, 희소 그래프 환경에서는 불필요한 계산이 많이 발생한다.

(3) 종합 결론

본 과제에서는 정점 500개와 간선 998개로 이루어진 희소 그래프에 대해 벨만-포드 알고리즘과 플로이드-워셜 알고리즘을 적용하고, 0번 정점에서 499번 정점까지의 최단 거리와 경로 및 수행 시간을 비교하였다. 실행 결과 두 알고리즘 모두 최단 거리 405를 출력하였으며, 최단 경로 또한 0 -> 1 -> 141 -> 97 -> 471 -> 472 -> 473 -> 474 -> 475 -> 336 -> 205 -> 268 -> 499로 동일하게 나타났다. 따라서 두 알고리즘이 동일한 최단 경로 문제에 대해 올바른 결과를 계산했음을 확인할 수 있었다.

성능 측면에서는 두 알고리즘 사이에 큰 차이가 나타났다. 벨만-포드 알고리즘은 약 0.000494초가 소요된 반면, 플로이드-워셜 알고리즘은 약 3.344579초가 소요되었다. 이는 두 알고리즘의 목적과 시간 복잡도 차이에서 비롯된다. 벨만-포드 알고리즘은 단일 출발점 최단 경로 알고리즘으로, 간선 완화를 기반으로 하며 시간 복잡도는 O(VE)이다. 특히 본 과제의 그래프처럼 간선 수가 정점 수에 비해 적은 희소 그래프에서는 탐색해야 할 간선의 수가 제한적이므로 효율적으로 동작할 수 있다.

반면 플로이드-워셜 알고리즘은 모든 정점 쌍 사이의 최단 거리를 계산하는 알고리즘으로, 시간 복잡도는 O(V^3)이다. 본 과제에서는 0번 정점에서 499번 정점까지의 최단 경로만 필요했음에도 불구하고, 플로이드-워셜은 모든 출발 정점과 도착 정점 조합에 대한 최단 거리를 함께 계산하였다. 따라서 희소 그래프에서는 실제 필요한 계산보다 훨씬 많은 연산을 수행하게 되었고, 그 결과 벨만-포드보다 현저히 긴 수행 시간이 나타났다.

결과적으로 본 실험은 최단 경로 알고리즘을 선택할 때 단순히 결과의 정확성뿐만 아니라, 문제에서 요구하는 최단 경로의 범위와 그래프의 특성을 함께 고려해야 함을 보여준다. 특정 출발점에서 특정 도착점까지의 최단 경로만 필요한 경우에는 벨만-포드와 같은 단일 출발점 최단 경로 알고리즘이 적합하다. 반면 모든 정점 쌍 사이의 최단 거리가 필요한 경우에는 플로이드-워셜 알고리즘이 유용할 수 있다. 즉, 최적의 알고리즘 선택은 그래프가 희소한지 밀집한지, 그리고 필요한 결과가 단일 경로인지 전체 정점 쌍의 경로인지에 따라 달라진다는 점을 확인하였다.